Chapter 1

Python for Machine Learning

1.1 Introduction

Machine Learning is a branch of Artificial Intelligence that enables computers to learn patterns from data and make decisions without being explicitly programmed for every situation. Today, Machine Learning is used in a wide variety of applications such as recommendation systems, image recognition, speech processing, medical diagnosis, fraud detection, and autonomous vehicles.

To develop Machine Learning applications, we need a programming language that is easy to learn, easy to use, and supported by a large collection of software libraries. Among the many programming languages available today, Python has become the most widely used language for Machine Learning.

Students of Computer Science often begin learning Machine Learning using Python because of its simplicity and readability. Python programs are generally shorter and easier to understand than programs written in many other languages. This allows learners to focus on understanding Machine Learning concepts rather than spending excessive time dealing with programming complexities.

In addition to its simple syntax, Python provides a rich ecosystem of libraries that simplify numerical computation, data processing, visualization, and Machine Learning model development. These libraries contain pre-written functions and classes that perform complex tasks efficiently. As a result, developers can build Machine Learning applications with relatively little code.

This chapter introduces the major Python libraries that are commonly used in Machine Learning experiments. The purpose of this chapter is to familiarize students with these libraries and demonstrate how they can be used through simple examples.

1.2 Why Python is Used for Machine Learning

Python has become the preferred language for Machine Learning for several reasons.

First, Python has a simple and readable syntax. Programs written in Python resemble ordinary English statements, making them easier to understand and maintain.

Second, Python is platform independent. A Python program developed on one operating system can generally be executed on another operating system without major modifications.

Third, Python provides a large collection of libraries specifically designed for scientific computing and Machine Learning.

Fourth, Python has a large community of users and developers. As a result, learners can easily find tutorials, examples, documentation, and solutions to common problems.

Finally, most modern Machine Learning frameworks and research tools provide direct support for Python. Therefore, learning Python is an essential step toward learning Machine Learning.

1.3 Python Libraries for Machine Learning

A Python library is a collection of pre-written functions and classes that can be imported and used in a program. Instead of writing every function from scratch, programmers can use libraries to perform tasks efficiently.

The most important libraries used in Machine Learning are:

1. NumPy
2. Pandas
3. Matplotlib
4. Scikit-learn

Each of these libraries serves a different purpose. The following sections discuss them in detail.

1.4 NumPy

NumPy stands for **Numerical Python**. It is one of the most important libraries used in Machine Learning, Data Science, Scientific Computing, and Artificial Intelligence applications.

Machine Learning algorithms work primarily with numerical data. Such data may represent marks of students, temperatures recorded by sensors, pixel values of images, stock market prices, medical measurements, or any other numerical information. Since Machine Learning involves large amounts of numerical computation, an efficient mechanism is required to store and process numerical values.

NumPy provides this functionality through a special data structure called an **array**. NumPy arrays allow large amounts of data to be stored and processed efficiently. Most Machine Learning libraries such as Scikit-learn, TensorFlow, and PyTorch internally use NumPy arrays for performing computations.

NumPy provides facilities for

- Creating arrays and matrices
- Performing arithmetic operations
- Statistical computations
- Matrix operations

- Data indexing and slicing
- Generating random numbers
- Handling multidimensional data

Because of these features, NumPy serves as the foundation for most Machine Learning applications.

1.4.1 Importing NumPy

Before using NumPy, it must be imported into the program.

```
import numpy as np
```

The keyword `as np` creates a shorter name for the library. Instead of writing `numpy.array()`, we can simply write `np.array()`.

1.4.2 What is an Array?

An array is a collection of elements stored together under a single name.

Consider the following collection of marks:

70, 75, 80, 85, 90

Instead of storing them in separate variables, they can be stored in a single array.

Arrays are extensively used in Machine Learning because datasets usually contain thousands or even millions of values.

1.4.3 Creating a One-Dimensional Array

The general syntax is

```
np.array(data)
```

where `data` contains the values to be stored.

```
import numpy as np

a = np.array([10,20,30,40])

print(a)
```

Output:

```
[10 20 30 40]
```

The variable `a` now stores a NumPy array containing four elements.

1.4.4 Creating a Two-Dimensional Array

Machine Learning datasets are often represented as tables consisting of rows and columns.

Such data can be stored using two-dimensional arrays.

```
import numpy as np

a = np.array([
[1,2,3],
[4,5,6]
])

print(a)
```

Output:

```
[[1 2 3]
 [4 5 6]]
```

Here, the array contains two rows and three columns.

1.4.5 Finding the Shape of an Array

The shape of an array specifies the number of rows and columns present.

The syntax is

```
array.shape
```

Example:

```
import numpy as np

a = np.array([
[1,2,3],
[4,5,6]
])

print(a.shape)
```

Output:

```
(2, 3)
```

This indicates that the array contains two rows and three columns.

1.4.6 Finding the Number of Dimensions

The syntax is

```
array.ndim
```

Example:

```
import numpy as np

a = np.array([[1,2,3],[4,5,6]])

print(a.ndim)
```

Output:

```
2
```

The output indicates that the array is two-dimensional.

1.4.7 Creating Arrays of Zeros

Sometimes Machine Learning programs require arrays whose values are initially zero.

The syntax is

```
np.zeros(size)
```

Example:

```
import numpy as np

a = np.zeros(5)

print(a)
```

Output:

```
[0. 0. 0. 0. 0.]
```

1.4.8 Creating Arrays of Ones

The syntax is

```
np.ones(size)
```

Example:

```
import numpy as np

a = np.ones(5)

print(a)
```

Output:

```
[1. 1. 1. 1. 1.]
```

1.4.9 Creating Sequential Numbers

The function `arange()` generates a sequence of values.

Syntax:

```
np.arange(start,stop,step)
```

Example:

```
import numpy as np

a = np.arange(1,10,2)

print(a)
```

Output:

```
[1 3 5 7 9]
```

1.4.10 Array Arithmetic Operations

NumPy allows arithmetic operations to be performed directly on arrays.

```
import numpy as np

a = np.array([1,2,3,4])

print(a + 5)
```

Output:

```
[6 7 8 9]
```

Each element is increased by 5.

Similarly,

```
print(a * 2)
```

Output:

```
[2 4 6 8]
```

Each element is multiplied by 2.

1.4.11 Finding the Sum of Elements

The syntax is

```
np.sum(array)
```

Example:

```
import numpy as np

a = np.array([10,20,30,40])

print(np.sum(a))
```

Output:

```
100
```

1.4.12 Computing the Mean

The mean represents the average value.

Syntax:

```
np.mean(array)
```

Example:

```
import numpy as np

a = np.array([10,20,30,40])

print(np.mean(a))
```

Output:

```
25.0
```

Mean values are frequently used in Machine Learning and data analysis.

1.4.13 Finding Maximum and Minimum Values

The syntax is

```
np.max(array)
np.min(array)
```

Example:

```
import numpy as np

a = np.array([10,20,30,40])

print(np.max(a))
print(np.min(a))
```

Output:

```
40
10
```

1.4.14 Computing Standard Deviation

Standard deviation measures the spread of data around the mean.

Syntax:

```
np.std(array)
```

Example:

```
import numpy as np

a = np.array([10,20,30,40])

print(np.std(a))
```

Output:

```
11.18
```

Standard deviation is frequently used in Machine Learning data preprocessing.

1.4.15 Indexing Elements

Individual elements can be accessed using indexes.

```
import numpy as np

a = np.array([10,20,30,40])

print(a[0])
print(a[2])
```

Output:

```
10
30
```

Indexing starts from 0.

1.4.16 Slicing Arrays

Slicing allows a portion of an array to be extracted.

```
import numpy as np

a = np.array([10,20,30,40,50])

print(a[1:4])
```

Output:

```
[20 30 40]
```

Slicing is commonly used when selecting subsets of data for analysis.

1.4.17 Importance of NumPy in Machine Learning

Almost every Machine Learning algorithm works on numerical data. Such data must be stored, manipulated, and processed efficiently. NumPy provides optimized data structures and mathematical functions that make these operations fast and convenient.

Many Machine Learning libraries including Scikit-learn internally use NumPy arrays. Therefore, understanding NumPy is essential before learning Machine Learning algorithms.

1.5 Pandas and DataFrames

Pandas is one of the most widely used Python libraries in Machine Learning and Data Science. Almost every Machine Learning project begins with a dataset, and Pandas provides powerful tools for storing, organizing, manipulating, and analyzing such data efficiently.

The name *Pandas* is derived from the term *Panel Data*. The library provides several useful data structures for handling data, among which the most important one is the **DataFrame**.

1.5.1 What is a DataFrame?

A DataFrame is a two-dimensional tabular data structure consisting of rows and columns. It is similar to a table in a database, a spreadsheet in Microsoft Excel, or a table stored in a CSV file.

Consider the following student dataset.

Roll No	Name	Marks
1	Anu	85
2	Rahul	90
3	Meera	88

In this table:

- Each horizontal entry is called a **row**.
- Each vertical entry is called a **column**.
- A row usually represents a single observation or record.
- A column represents a particular attribute or feature of the data.

Most Machine Learning algorithms expect data to be organized in tabular form. Therefore, DataFrames are extensively used for storing and processing datasets in Machine Learning applications.

1.5.2 Importing Pandas

Before using Pandas, it must be imported into the Python program.

```
import pandas as pd
```

The keyword `as pd` assigns a short name to the library, making it easier to access its functions.

1.5.3 Creating a DataFrame

A DataFrame can be created from dictionaries, lists, arrays, or external files. The general syntax is

```
pd.DataFrame(data)
```

where `data` contains the information to be stored in tabular form.

The following example creates a DataFrame from a Python dictionary.

```
import pandas as pd

data = {
'Name':['Anu','Rahul','Meera'],
'Marks':[85,90,88]
}

df = pd.DataFrame(data)

print(df)
```

Output:

```
Name  Marks
0    Anu     85
1  Rahul     90
2  Meera     88
```

In the above example, the keys `Name` and `Marks` become column names, while the values associated with them form the rows of the DataFrame. The variable `df` stores the entire table and can be used for various data analysis operations.

1.5.4 Reading Data from a CSV File

In real-world Machine Learning applications, datasets are usually stored in files rather than being entered manually. One of the most common file formats is the CSV (Comma Separated Values) format.

A CSV file stores data in tabular form, where the values in each row are separated by commas. For example, a file named `students.csv` may contain the following data:

```
Name,Marks
Anu,85
Rahul,90
Meera,88
```

Pandas provides the `read_csv()` function to read such files and convert them into a DataFrame.

```
import pandas as pd

df = pd.read_csv("students.csv")

print(df)
```

The function reads the contents of the file and stores them in the DataFrame `df`. This is one of the most frequently used functions in Machine Learning because datasets are commonly distributed in CSV format.

1.5.5 Displaying the First Few Rows

Large datasets may contain thousands of rows. Displaying the entire dataset is often impractical. Pandas provides the `head()` function to display the first few rows of a DataFrame.

```
print(df.head())
```

By default, the function displays the first five rows of the dataset. This helps users quickly inspect the structure and contents of the data.

1.5.6 Obtaining Statistical Information

Before applying Machine Learning algorithms, it is often useful to understand the characteristics of the dataset. Pandas provides the `describe()` function for this purpose.

```
print(df.describe())
```

The `describe()` function generates summary statistics for numerical columns, including:

- Count of values
- Mean (average)
- Standard deviation
- Minimum value
- Maximum value
- Quartiles

These statistics help in understanding the distribution and variability of the data and are frequently used during data preprocessing and exploratory data analysis.

1.6 Matplotlib

In Machine Learning and Data Science, understanding data is just as important as developing algorithms. Numerical values stored in tables often do not reveal useful information immediately. Graphical representations help us understand patterns, trends, distributions, and relationships within data. This process is known as **data visualization**.

Matplotlib is one of the most widely used Python libraries for data visualization. It provides functions for creating a variety of graphs and charts. Using Matplotlib, we can represent numerical data visually and gain insights that may not be obvious from tables alone.

Data visualization is important in almost every stage of a Machine Learning project. Before training a model, graphs help us understand the dataset. After training, graphs help us analyze and present the results.

Matplotlib contains several modules. The most commonly used module is `matplotlib.pyplot`, which provides functions for creating and displaying graphs.

1.6.1 Importing Matplotlib

Before using Matplotlib, it must be imported into the Python program.

```
import matplotlib.pyplot as plt
```

The keyword `plt` is simply a short name assigned to the module. Instead of writing `matplotlib.pyplot` repeatedly, we use the shorter name `plt`.

1.6.2 Creating a Simple Line Graph

A line graph is used to show how a quantity changes with respect to another quantity, usually time.

The general syntax is

```
plt.plot(x,y)
```

where

- `x` contains values on the horizontal axis.
- `y` contains values on the vertical axis.

Example:

```
import matplotlib.pyplot as plt

days = [1,2,3,4,5]
sales = [100,120,140,130,160]

plt.plot(days,sales)

plt.show()
```

The graph displays how sales vary across different days.

1.6.3 Adding a Title

Every graph should have a meaningful title so that viewers can understand what the graph represents.

Syntax:

```
plt.title("Title")
```

Example:

```
plt.title("Daily␣Sales")
```

1.6.4 Labeling Axes

Axes should be labeled properly.

Syntax:

```
plt.xlabel("Label")
plt.ylabel("Label")
```

Example:

```
plt.xlabel("Days")
plt.ylabel("Sales")
```

1.6.5 Drawing a Bar Chart

A bar chart is used for comparing values belonging to different categories.

The general syntax is

```
plt.bar(categories,values)
```

Example:

```
import matplotlib.pyplot as plt

subjects = ["Maths",
            "Physics",
            "Chemistry"]

marks = [80,92,75]

plt.bar(subjects,marks)

plt.title("Marks␣in␣Subjects")

plt.xlabel("Subjects")
plt.ylabel("Marks")

plt.show()
```

In a bar chart, the height of a bar represents the corresponding value.

1.6.6 Drawing a Horizontal Bar Chart

Sometimes category names are long and difficult to display vertically. In such situations, a horizontal bar chart is useful.

Syntax:

```
plt.barh(categories,values)
```

Example:

```
import matplotlib.pyplot as plt

subjects = ["Maths",
            "Physics",
            "Chemistry"]

marks = [80,92,75]

plt.barh(subjects,marks)

plt.show()
```

1.6.7 Drawing a Pie Chart

A pie chart shows how a total quantity is divided among different categories.

The general syntax is

```
plt.pie(values,labels=labels)
```

Example:

```
import matplotlib.pyplot as plt

marks = [40,30,20,10]

subjects = ["Maths",
            "Physics",
            "Chemistry",
            "English"]

plt.pie(marks,
        labels=subjects)

plt.show()
```

Each slice of the circle represents a percentage of the whole.

1.6.8 Displaying Percentages in Pie Charts

Syntax:

```
plt.pie(values,
        labels=labels,
        autopct='%1.1f%%')
```

Example:

```
plt.pie(marks,
        labels=subjects,
        autopct='%1.1f%%')

plt.show()
```

The percentages are displayed on the pie chart.

1.6.9 Drawing a Scatter Plot

A scatter plot is one of the most important graphs used in Machine Learning.

It helps us study the relationship between two variables.

Syntax:

```
plt.scatter(x,y)
```

Example:

```
import matplotlib.pyplot as plt
```

```
hours = [1,2,3,4,5]
marks = [40,50,60,70,80]

plt.scatter(hours,marks)

plt.xlabel("Hours Studied")
plt.ylabel("Marks")

plt.show()
```

If the points generally move upward, it indicates a positive relationship between the variables.

1.6.10 Drawing a Histogram

A histogram is used to study the distribution of numerical data.

Histograms are widely used during data preprocessing.

Syntax:

```
plt.hist(data)
```

Example:

```
import matplotlib.pyplot as plt

marks = [45,50,60,70,72,
         75,80,82,90,95]

plt.hist(marks)

plt.show()
```

The histogram groups values into intervals and shows how frequently each interval occurs.

1.6.11 Drawing Multiple Graphs

More than one graph can be displayed in the same figure.

Example:

```
import matplotlib.pyplot as plt

x = [1,2,3,4]

y1 = [10,20,30,40]
y2 = [15,25,35,45]

plt.plot(x,y1)

plt.plot(x,y2)

plt.show()
```

1.6.12 Saving a Graph

Instead of displaying a graph, it can be saved as an image file.

Syntax:

```
plt.savefig("graph.png")
```

Example:

```
plt.plot([1,2,3],[10,20,30])

plt.savefig("graph.png")
```

The graph will be stored in the current directory.

1.7 Scikit-learn

Scikit-learn is one of the most popular Machine Learning libraries available in Python.

Machine Learning algorithms involve complex mathematical computations. Implementing these algorithms manually requires a considerable amount of programming effort. Scikit-learn provides ready-made implementations of many Machine Learning algorithms.

As a result, developers can focus on understanding Machine Learning concepts rather than implementing algorithms from scratch.

Scikit-learn supports

- Classification
- Regression
- Clustering
- Data Preprocessing
- Model Evaluation
- Feature Selection
- Dimensionality Reduction

1.7.1 Important Terms in Machine Learning

Before learning Scikit-learn, it is important to understand a few basic terms.

Dataset

A dataset is a collection of observations.

Example:

Hours	Attendance	Result
2	80	Pass
4	90	Pass
1	60	Fail

Features

The input variables are called features.
In the above dataset

- Hours Studied
- Attendance

are features.

Target Variable

The output variable that must be predicted is called the target variable.
In the above example, Result is the target variable.

Model

A Machine Learning algorithm that learns from data is called a model.

1.7.2 Loading a Dataset

Scikit-learn provides several sample datasets.
Example:

```
from sklearn.datasets import load_iris

iris = load_iris()

print(iris.data[:5])
```

Output:

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.0 1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.0 3.6 1.4 0.2]]
```

1.7.3 Features and Target Variables

In Scikit-learn,

```
X = iris.data
y = iris.target
```

Here,

- `X` contains feature values.
- `y` contains target values.

1.7.4 Creating a Machine Learning Model

Example:

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier()
```

The statement creates a Decision Tree classification model.

1.7.5 Training a Model

Training means allowing the model to learn patterns from data.

Syntax:

```
model.fit(X,y)
```

Example:

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()

X = iris.data
y = iris.target

model = DecisionTreeClassifier()

model.fit(X,y)

print("Training␣Completed")
```

Output:

```
Training Completed
```

1.7.6 Making Predictions

After training, the model can predict outputs for new data.

Syntax:

```
model.predict(data)
```

Example:

```
prediction = model.predict(
[[5.1,3.5,1.4,0.2]]
)

print(prediction)
```

The model predicts the class corresponding to the given input.

1.7.7 Why Scikit-learn is Important

Almost every Machine Learning experiment performed in this laboratory will use Scikit-learn.

The library provides efficient implementations of Machine Learning algorithms and evaluation methods. By combining Pandas for data handling, NumPy for numerical computation, Matplotlib for visualization, and Scikit-learn for model development, complete Machine Learning applications can be developed with relatively small amounts of code.

1.8 Summary

Python has become the standard programming language for Machine Learning because of its simplicity and extensive collection of libraries. NumPy provides efficient numerical computations, Pandas supports data manipulation and analysis, Matplotlib enables graphical visualization, and Scikit-learn offers implementations of Machine Learning algorithms.

A good understanding of these libraries is essential for performing Machine Learning experiments. In the subsequent chapters of this manual, these libraries will be used extensively to implement classification, regression, clustering, and other Machine Learning techniques.